Introduction to Neural Network

DR. SATISH R. SANKAYE

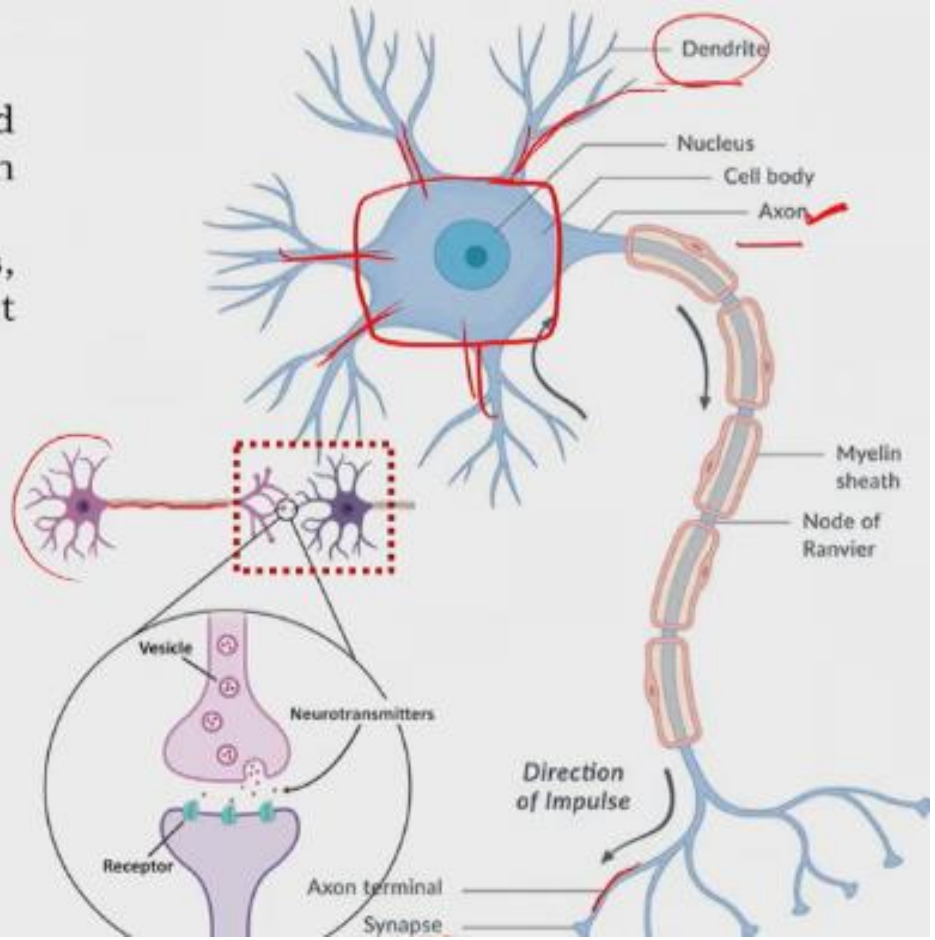
Biological Inspiration for Neural Network

Human Brain as Inspiration

- The human brain consists of billions of interconnected neurons that process and transmit information through electrochemical signals.
- Each neuron receives inputs through dendrites, integrates them in the cell body, and sends an output signal through the axon to other neurons via synapses.

Axon: Thin fiber-like projection of a neuron. This axon carry the electrical signal and sends information to another neurons.

Synapses: A synapse is a connection point between the axon terminal of one neuron and the dendrite of another neuron.
The electrical signal from the axon is converted into a chemical signal that crosses the small synaptic gap.

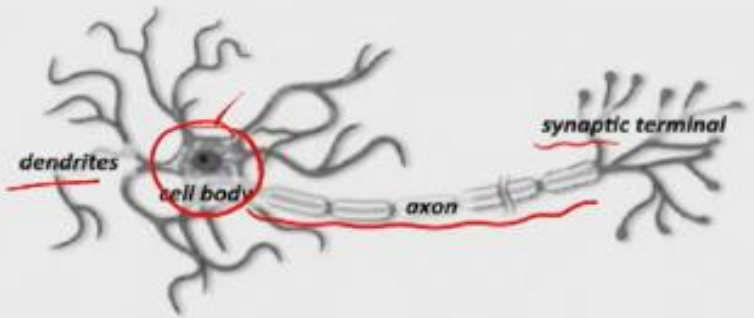


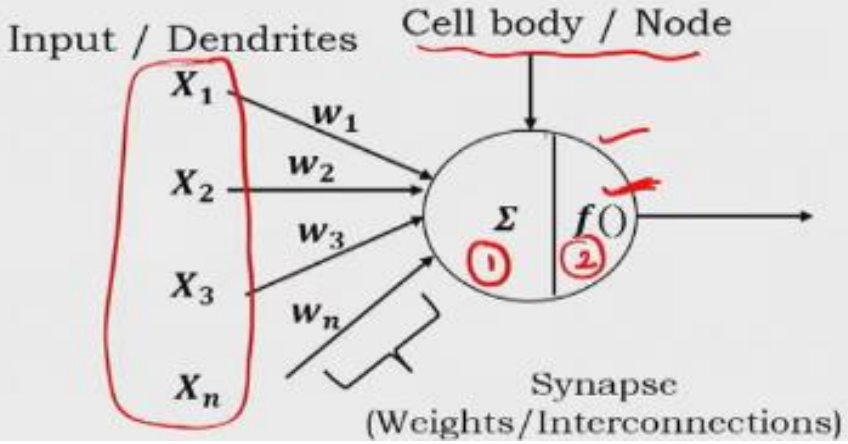
Artificial Neuron for Neural Network

Biological Neuron		Artificial Neuron
Dendrites	→	Inputs
Cell body	→	Node / Summation unit
Firing	→	Activation Function
Axon	→	Connection/Output
Synapse	→	Weights

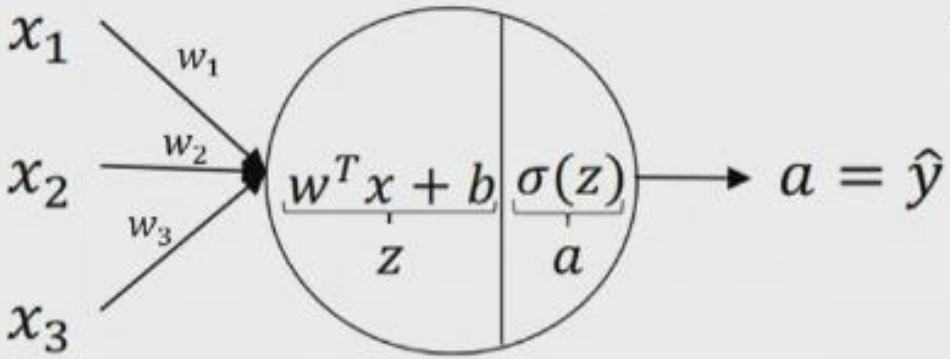
Artificial Analogy

- Artificial Neural Networks (ANNs) are computational models inspired by the structure and functioning of biological neurons.
- Each artificial neuron receives multiple inputs, multiplies them by their connection weights, sums them, and applies an activation function — mimicking how a biological neuron "fires" when sufficiently stimulated.





Mathematical Representation of Neuron



Suppose you're predicting if a student will pass based on **3 features**:

x_1: *Hours studied,*
x_2: *Attendance percentage,*
x_3: *Previous grade*

Lets say: $x = \begin{bmatrix} 8 \\ 90 \\ 75 \end{bmatrix}, \mathrm{w} = \begin{bmatrix} 0.4 \\ 0.3 \\ 0.2 \end{bmatrix}, b = 1$

Then: $x \cdot w$ *is not possible. It has to be* $w^T \cdot x$

$$z = w^T x + b$$

$$a = \sigma(z)$$

Without bias

With bias

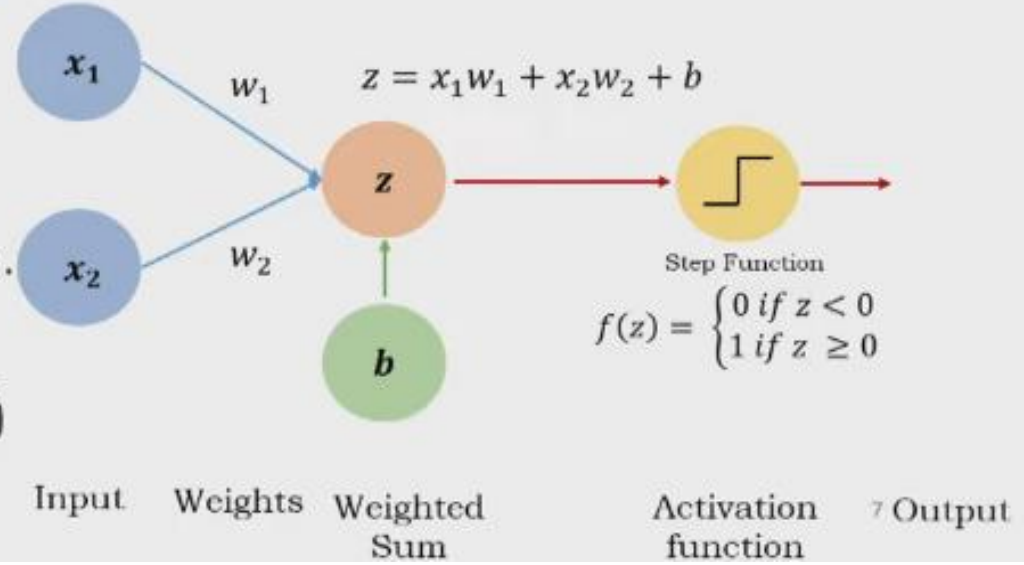
- *The value of the b has to be low to make the model converge & not stop progress.*
- *The Activation function is used to make a signal trigger or not.*

Perceptron

- A **Perceptron** is the simplest and oldest form of a neural network, introduced by Frank Rosenblatt in 1958.
- It is an algorithm used for **supervised learning of binary classifiers** — that is, it can classify data into **two classes** (e.g., 0 or 1, Yes or No, True or False).
- A perceptron consists of an **input layer** and an **output layer** (no hidden layers). It is mainly used to model simple **logical operations** such as **AND, OR, and NOR**, which have **binary outputs**.

Working of Perceptron:

Step 1: Multiply each input x_i by its corresponding weight w_i and add the bias b.
Step 2: Compute the Weighted Sum z.
Step 3: Pass the weighted sum through an activation function (usually a step function) to decide the output.



Example:

Component	Example 1 – Spam Detection
Input Features	x1= number of suspicious words; x2= % capital letters in subject; x3= number of links | **Input Values** | $x_1 = 4,\ x_2 = 0.2,\ x_3 = 2$|
Weights	w1=0.8, w2=0.5, w3=0.3
Bias	b=0.5 The bias helps shift the decision threshold so that the neuron can classify emails correctly even when all feature values are low.
Summation Function	S=(4*0.8) + (0.2*0.5) + (2*0.3) + 0.5 = 4.4
Activation Function	Threshold θ=3.0 If S≥3.0 → Output = 1 (Spam), else 0 (Not Spam)
Output	1 (Spam) **Interpretation:** Since S = 4.4 > 3.0, the email is classified as **Spam (1)**. This means the combination of features (many suspicious words and links) strongly indicates spam behavior.
Learning Algorithm	If spam email is misclassified as "Not Spam," increase w1,w2,w3 and bias slightly.

Perceptron Learning Algorithm

2. Repeat for each epoch:

For each training sample

1. Compute the weighted sum:

   $\text{sum} = \sum_{i=1}^{n} w_i^T . x_i + b$

2. Pass the weighted sum to the activation function

   $$\gamma = \begin{Bmatrix} 1 \; if \; f(x) \geq \theta \\ 0 \; if \; f(x) < \theta \end{Bmatrix}, \; \theta \; is \; threshold$$

3. Calculate error: $target - computed$

4. Update the weights and bias

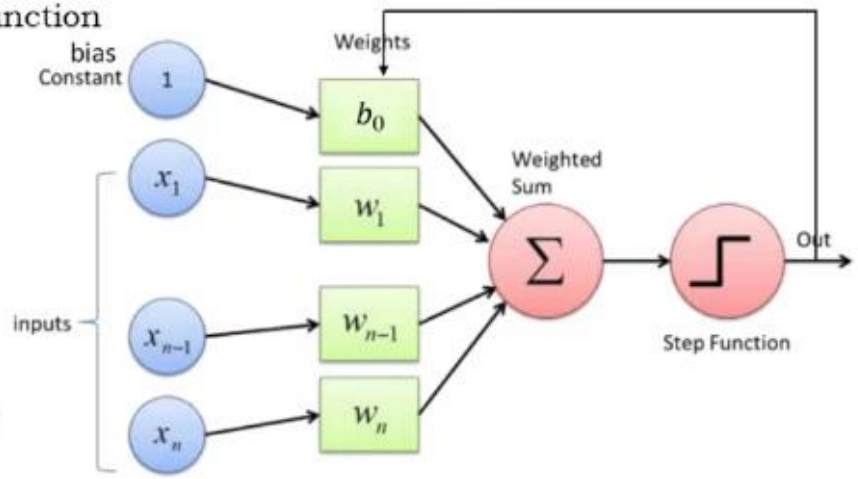
   $w_{new} \leftarrow w_{old} + \Delta w$

   $\Delta w = \eta(target - computed)x_i$

   $bias_{new} \leftarrow bias_{old} + \eta(target - computed)$

1.Initialize:

1. Set weights $w_1, w_2, \ldots . w_n$ and the bias b_0 to the random values.
2. Choose the learning rate η



Perceptron Learning Algorithm

2. Repeat for each epoch:

For each training sample

1. Compute the weighted sum:

   $\text{sum} = \sum_{i=1}^{n} w_i^T . x_i + b$

2. Pass the weighted sum to the activation function

   $\gamma = \begin{Bmatrix} 1 \; if \; f(x) \geq \theta \\ 0 \; if \; f(x) < \theta \end{Bmatrix}, \; \theta \; is \; threshold$

3. Calculate error: $target - computed$

4. Update the weights and bias

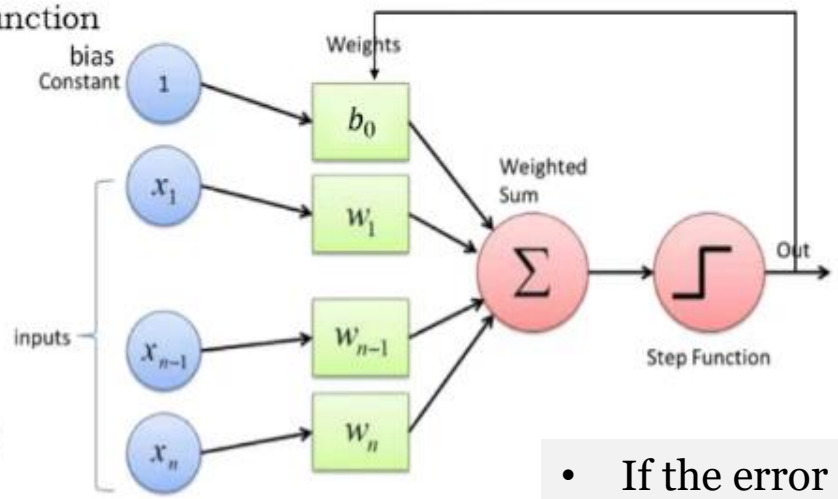
   $w_{new} \leftarrow w_{old} + \Delta w$

   $\Delta w = \eta(target - computed)x_i$

   $bias_{new} \leftarrow bias_{old} + \eta(target - computed)$

1.Initialize:

1. Set weights $w_1, w_2, \ldots . w_n$ and the bias b_0 to the random values.
2. Choose the learning rate η



Let's Solve One Problem:

Inputs: x = $[x_1, x_2]$ = [1.0, 2.0]

Target: y = 1

Weights: w = $[w_1, w_2]$ = $[0.2, -0.5]$

Bias: b= 0.1

Learning rate η = 0.1

Prediction rule

if computed value is ≤ 0 *then* 0 *otherwise* 1

- If the error is too small, increase the weight.
- If the error is too large, decrease the weight.

Steps to Solves the Problem

1. Computer Z : w1x1 + w2x2 + b , z = -0.7
2. Based on Activation function Calculate Target : y' = 1 [As z <=0, 1 otherwise 0]
3. Error : target – computed , so error : 1 – 0 = 1

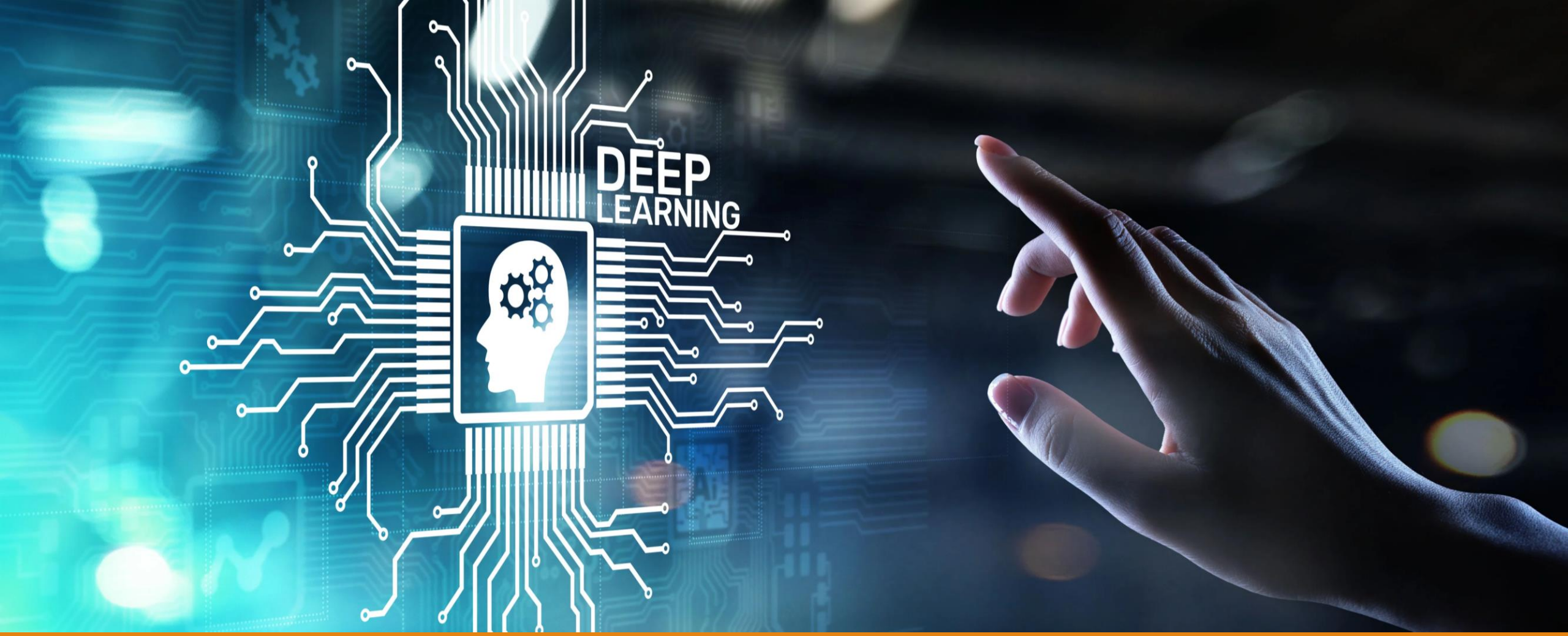
Now update the weight as Error is present:

4. Compute Wn = Wo – Δ(W) and Δ(Wn) = η (Error) Xn
5. W1 = 0.3 and W2 = -0.3 and B = 0.2

Now repeat above step 1,2 and 3, as error present continue step 4 and 5 too.

W1 = 0.4 and W2 = -0.1 and B = 0.3

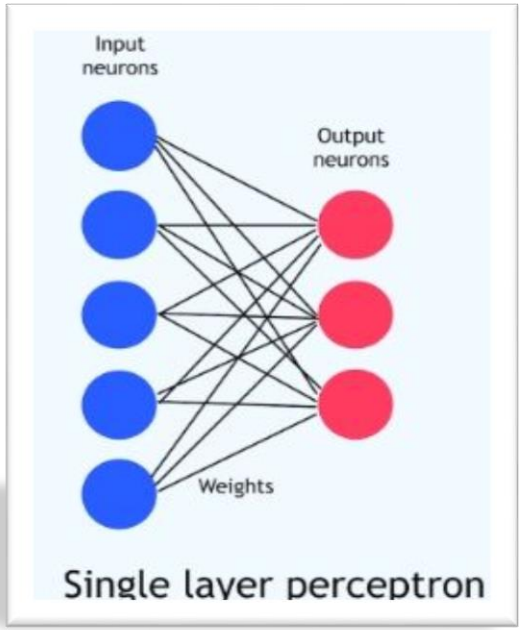
- If We get the perfect values of the weight and bias we can check it for the unknow: Say x1 = 2 and x2 = 1 , Find the result .



Types of Perceptron

Single Layer Perceptron & Multilayer Perceptron

Types of Perceptron: Single-Layer



$$w_{new} \leftarrow w_{old} + \Delta w$$

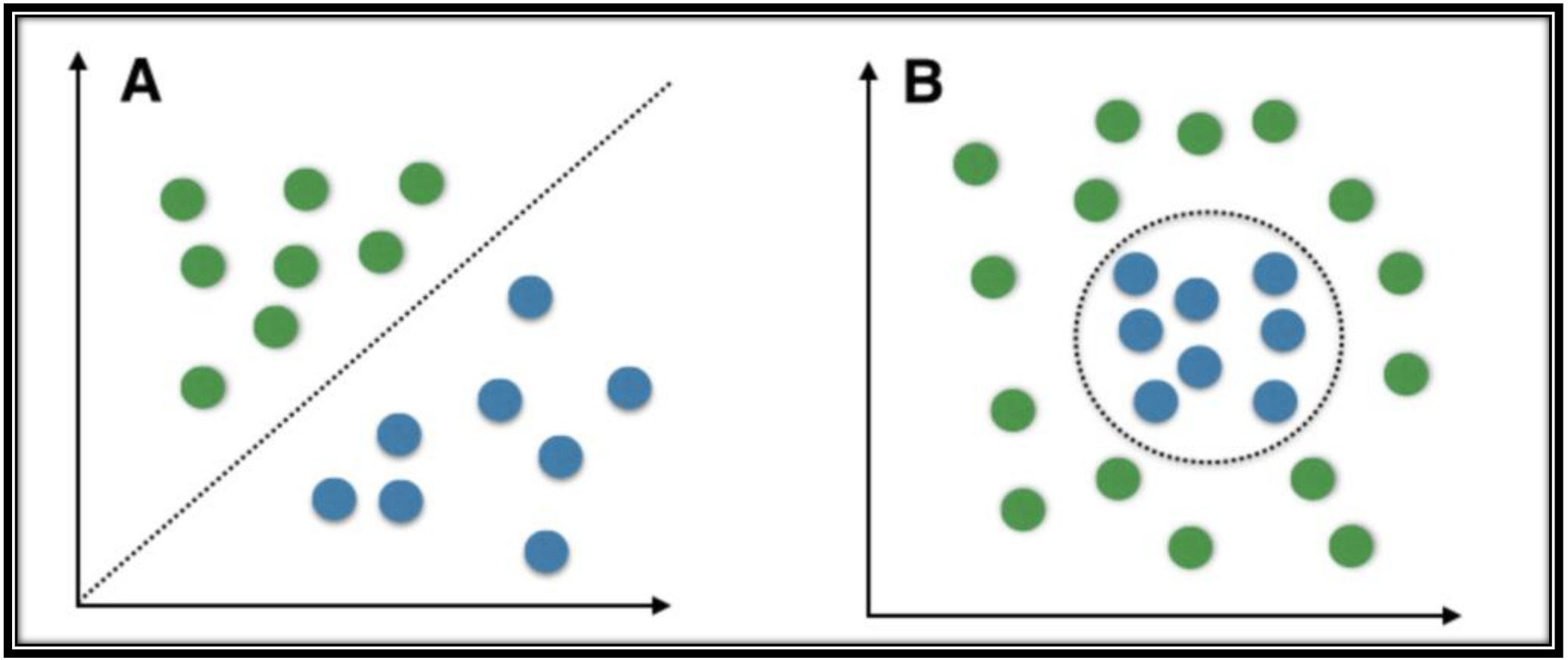
$$\Delta w = \eta(target - computed)x_i$$

$$bias_{new} \leftarrow bias_{old} + \eta(target - computed)$$

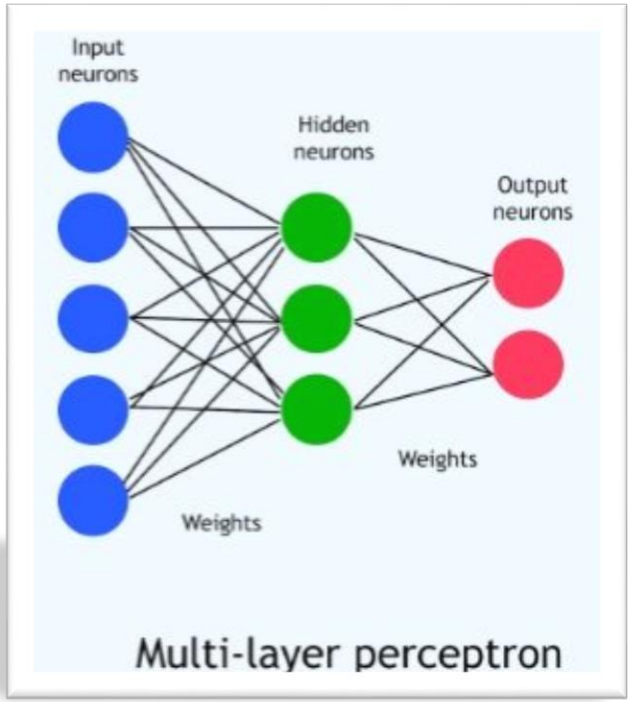
Single Layer Perceptron:

- It is the simplest form of **ANN** (Artificial Neural Network).
- It works only on **Linearly separable Patterns.**
- The architecture consist of only an **Input Layer** and an **Output Layer. (No Hidden Layer)**
- The activation function used is typically a **step or threshold function.**
- It is more suitable for the **Binary Classification (1 or 0).**
- The Model uses the **Perceptron Learning Algorithm rule.**

Types of Perceptron: Single Layer



Types of Perceptron: Multi-Layer



Multi Layer Perceptron Model:

- It is capable to works on both **Linearly separable Patterns** as well as **Non Linearly Separable Patterns.**
- The architecture consist of an **Input Layer** and an **Output Layer** with **One or More Hidden Layer.**
- The activation function used are typically a non-linear such as **tanh, ReLU, sigmoid function and so on.**
- It is suitable for the **Classification and Regression Task.**
- The Model is trained using the **back-propagation algorithms with gradient descent or its variants.**
- It is mainly used for solving **real world complex problems.**

Single Layer Perceptron: Problem

X1	X2	Target
0	0	0
0	1	1
1	0	1
1	1	1

Learning Rate : η = 0.2

Threshold : θ = 0.5

>= 0.5 -- 1

< 0.5 -- 0

z = x1w1 + x2w2

X1	X2	W1	W2	Computed	Target	Error	Δw1	Δw2
0	0	0.1	0.3					

Limitation: Single Layer Perceptron

The following are the Limitation of Single Layer Perceptron:

1. **Non-Linearity:** Fails to learn patterns where classes aren't separable by a straight line, exemplified by the XOR gate
2. **Binary Output:** Can only classify data into two categories (0 or 1), making it unsuitable for multi-class problems without extensions.
3. **No Hidden Layers:** Its single-layer structure prevents it from learning hierarchical or abstract features, limiting its representational power.
4. **Data Sensitivity:** Requires well-prepared, scaled, and normalized data; performance degrades with noisy or poorly scaled inputs.
5. **Limited Complexity:** Too simple for complex tasks in vision, NLP, or sequence modeling, requiring deeper networks.

Implementation of Single Layer Perceptron

The following Python Code can be used to implement Single Layer Perceptron:

```
import  numpy as np
# Given Data
x  = np.array([1.0, 2.0])        # inputs
y = 1                            # target
w = np.array([0.2, -0.5])        # Initial weight
b = 0.1                          # Initial Bias
lr  = 0.1                        # Learning Rate
```